Maturaarbeit

Johannes Rappazzo

September 2026

1 Einleitung

Schach ist ein altes Spiel. Dazu ist es ein komplexes Spiel mit unglaublich vielen Möglichkeiten und legalen Stellungen. Tatsächlich gibt es $2*10^{43}$ mögliche legale Schachstellungen [3]. Dies ist eine enorme Anzahl Stellungen. Wenn man nun einen perfekten Schachcomputer bauen möchte, stellt diese Anzahl Stellungen eine besondere Herausforderung. Wenn man nämlich ein Spiel wie Tic-Tac-Toe automatisieren will, kann man einfach den gesamten Spielverlauf analysieren und dadurch den perfekten Weg finden. Allerdings geht dies bei Schach nicht, da es einfach zu viele Stellungen sind. Dies hat die Informatik jedoch nicht davon abgehalten, das Schachspiel zu knacken. Denn schon 1996, schon gewann der erste Schach-Bot gegen einen amtierenden Weltmeister. [**Wikikasparov**]. Um dieses Problem der Anzahl Stellungen zu lösen fand die Informatik einige geniale Lösungen, welche ich mir in dieser Arbeit zu nutzen machen konnte beim Bau meines eigenen Schachcomputers.

1.1 Motivation/persönlicher Bezug

Ich spiele schon sehr lange Schach. Um ehrlich zu sein kann ich mich nicht daran erinnern, nicht Schach gespielt zu haben. Meinem ersten Schachklub bin ich mit 6 Jahren beigetreten. Dies war natürlich ein Kinderschachklub. Seit dann habe ich mich in das Schachspiel verliebt und spiele auch bis heute noch aktiv Schach. Dabei ist das Nutzen von Schachcomputer ein aktiver Bestandteil meines Schachalltags. Ich verwende ihn für die Analyse meiner Partien, für das Training sowie für die Eröffnungsvorbereitung. Dabei benutze ich ihn vorallem für das letzte. Tatsächlich habe ich ein wenig ein Ruf dafür, dass ich immer sehr gut vorbereitet bin in der Eröffnung. Dies liegt an den unzähligen Stunden die ich schon beim analysieren von Eröffnungsvarianten verbracht habe. Somit war ich schon lange intensiv im Kontakt mit Schachcomputern. Dabei hat es mich schon immer gewundert, wie diese Programme eine Stellung nehmen können und einen für den Menschen scheinbar perfekten Zug generieren können. Gleichzeitig habe ich mich schon länger für programmieren interessiert und habe auch nach einer Ausrede gesucht, malwieder mehr zu programmieren. Dies hat dann dieses Projekt angeboten.

1.2 Fragestellung und Ziel

In dieser Arbeit habe ich mich darin versucht, selber einen klassischen Schachcomputer mit C++ zu programmieren. Dabei lautet die Fragestellung:

Welche Elostärke kann ein selbst programmierter Schachcomputer erreichen mit alpha-beta Suche und klassischer Stellungsbewertung und welche Bestandteile sind dabei am wichtigsten?

Dabei habe ich zuerst merheitlich selber[1] einen Schachcomputer entwickelt. Danach habe ich systematisch Bestandteile aus der Engine gelöscht und die nun verringerte Version gegen die komplette Version antreten lassen um herauszufinden, welche Bestandteile am wichtigsten sind.

2 Theorie

2.1 Schachregeln

2.1.1 Bauer

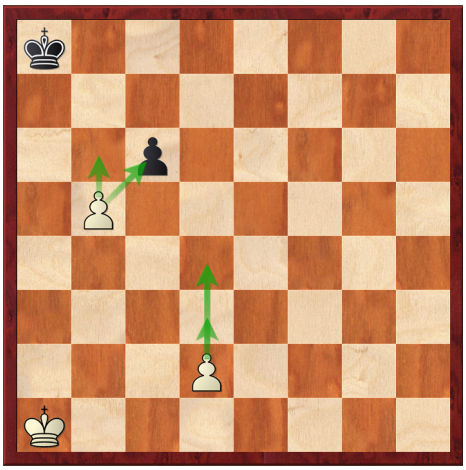


Abbildung 1: Bauernzüge

Bauern können jeweils ein Feld nach vorne ziehen. Ausser beim deren ersten Zug. Da können sie auch zwei Felder nach vorne ziehen. Schlagen können sie jedoch nicht nach vorne. Dies können sie nur diagonal. Wichtig ist noch, dass Bauern nur nach vorne ziehen dürfen. Es gibt noch weiter Regeln zu Bauern, diese werden jedoch separat behandelt.

[1] Ich habe auch Claude als Hilfsmittel benutzt. Mehr dazu allerdings im entsprechenden Kapitel

2.1.2 Läufer

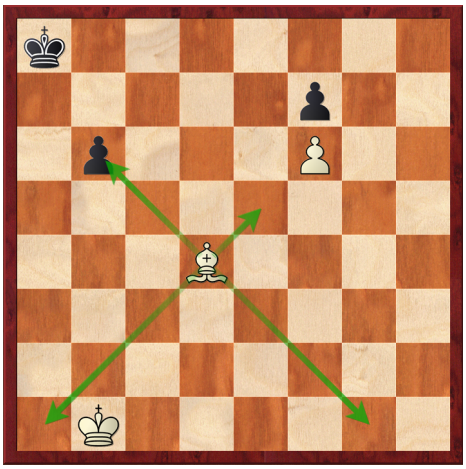


Abbildung 2: Läuferzüge

Läufer können diagonal in alle Richtungen so weit wie möglich ziehen. Dabei ist am Läufer speziell, dass er nur jeweils auf einer Farbe ziehen kann

2.1.3 Turm

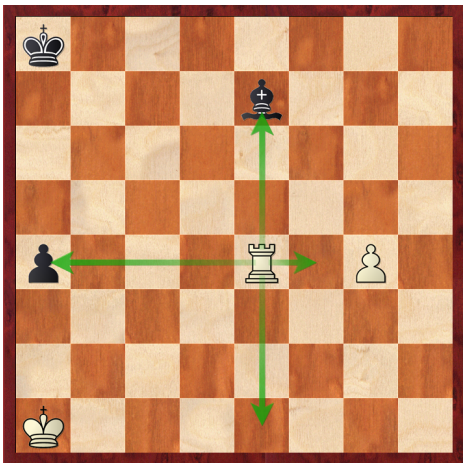


Abbildung 3: Turmzüge

Türme können horizontal sowie vertikal ziehen so weit wie möglich.

2.1.4 Dame

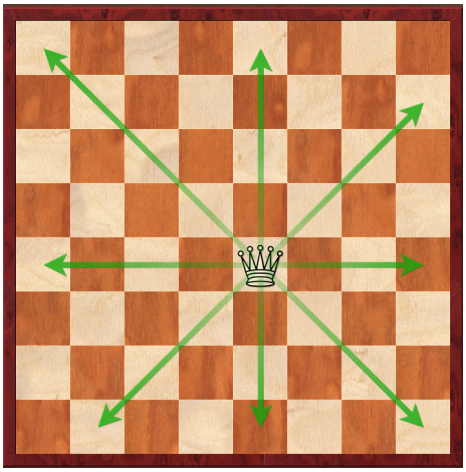


Abbildung 4: Enter Caption

Die Dame ist wie eine Mischung aus Turm und Läufer. Sie kann horizontal, vertikal sowie diagonal so weit wie möglich in alle Richtungen gehen.

2.1.5 Springer

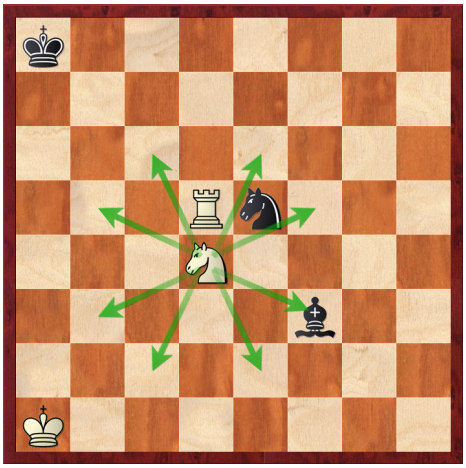


Abbildung 5: Enter Caption

Der Springer zieht in einer L-Form. Dabei geht er immer zwei Felder in eine Richtung und danach ein Feld in eine dazu senkrechte Richtung. Der Springer

ist zudem einzigartig, da er die einzige Figur ist, die über andere Figuren ziehen kann.

2.1.6 König

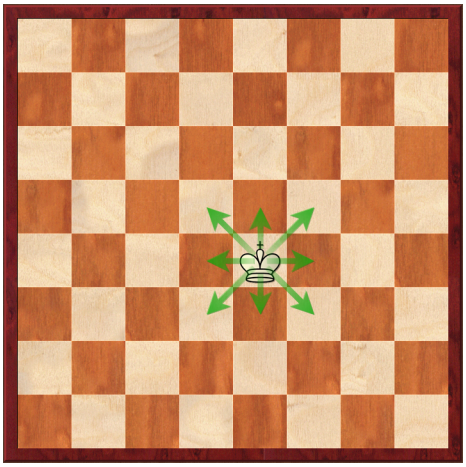


Abbildung 6: Enter Caption

Der König kann jeweils ein Feld in alle Richtungen ziehen. Man könnte deshalb denken, dass der König eine unwichtige Figur ist. Jedoch ist er dennoch die wichtigste Figur auf dem Brett. Dies liegt daran, dass das Ziel von Schach ist, den gegnerischen König schachmatt zu setzen. Ein König ist Schachmatt, wenn nach allen eigentlich legalen Zügen der König immernoch im Schach steht. Im Schach stehen bedeutet einfach, dass der König angegriffen ist. Auf ein Schach muss zudem reagiert werden. Wenn man dies nicht kann, ist es Schachmatt. Das bedeutet auch, dass der König nicht auf ein angegriffenes Feld ziehen darf.

2.1.7 Spezialregeln

2.1.7.1 Rochade Wenn der König sowie der Turm noch nicht gezogen haben, es keine Figuren zwischen dem Turm und dem König gibt, der König nicht im Schach steht und der König im Verlauf der Rochade nicht durch ein Schach ziehen würde, darf man eine Rochade durchführen. Dabei geht der König zwei Felder zur Seite und der Turm springt über den König auf die andere Seite. Man darf keine gegnerischen Figuren schlagen bei der Rochade. Es gibt die kurze Rochade sowie die lange Rochade. Jedoch funktionieren beide nach den oben genannten Regeln.

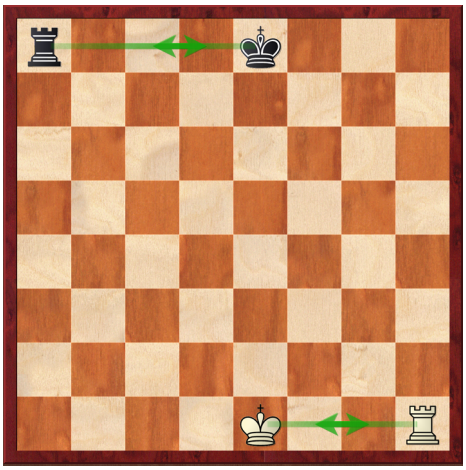


Abbildung 7: Rochade

2.1.7.2 En passent Wenn ein Bauer zwei Felder nach vorne zieht und ein gegnerischer Bauer gerade neben dran ist, darf dieser Bauer für genau einen Zug vom angrenzenden, gegnerischen Bauern geschlagen werden. Dabei tut man so, als hätte der Bauer nur ein Feld gezogen.

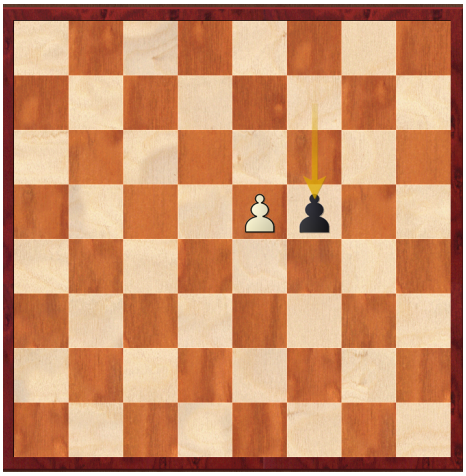


Abbildung 8: Der Bauer auf e5 darf hier den Bauern auf f5 nehmen

2.1.7.3 Umwandlung Wenn ein Bauer die grundreihe erreicht, muss er in eine beliebige Figur der eigenen Farbe umgewandelt werden welche nicht ein Bauer oder König ist.

2.1.7.4 diverse Remis Regeln

2.1.7.4.1 Patt Ein Patt liegt vor, wenn ein Spieler keine legalen Züge mehr hat und der König nicht im Schach steht.

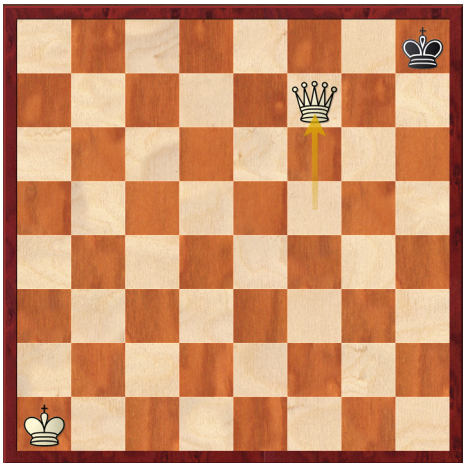


Abbildung 9: Ein Patt mit schwarz am Zug

2.1.7.4.2 dreifache Stellungswiederholung Wenn die genau gleiche Stellung dreimal vorkommt, ist es remis. Dabei muss die Wiederholung nicht am Stück passieren. Dazu muss die Stellung identisch sein. Das heisst, wenn beim ersten mal ein en-Passent möglich war, zählt es nicht als eine Wiederholung.

2.1.7.4.3 50 Züge Regel Wenn für 50 Züge keine Figur geschlagen wurde und kein Bauer gezogen hat, ist es auch remis. Diese Regel existiert, damit Partien nicht für immer weitergehen können.

2.2 eine kurze Geschichte zu Schachcomputern

2.2.1 deep Blue

Schon Alan Turing und Claude Shannon haben versucht, Schachprogramme zu entwickeln [4]. Allerdings war es erst 1996, als IBMs Deep Blue zum ersten mal einen amtierenden Weltmeister schlagen konnte. Diese Partie war ein wichtiger Durchbruch in der Welt von Schachcomputer.

2.2.2 Alpha zero, AI und NNUEs

Traditionelle Schachcomputer funktionieren komplett algorithmisch. Für lange Zeit war dies auch der einzige Weg, um einen Schachcomputer zu bauen. Jedoch wurde 2017 alles auf den Kopf gedreht, als Googles Alpha Zero den damals besten Schachcomputer Stockfish in einem Match mit 28 Siegen, 72 unentschieden und keinem Verlust schlug. Während Stockfish damals noch eine traditionelle Engine war, basierte Alpha Zero auf einem neuronalen Netzwerk

welches mit sich selber spielte und so sich selbst trainierte[2]. Während damals Stockfish noch komplett algorithmisch funktionierte, verwendet Stockfish heute NNUE für seine Stellungsbewertungsfunktion. Traditionellerweise benutzt man vorsichtig abgemessene Formeln, welche die Stellung nach ausgewählten Bewertungskriterien evaluieren[5]. So habe ich es auch in meiner Arbeit gemacht.

2.2.3 positive Einflüsse von Schachcomputern auf das menschliche Spiel

Es ist immer gut, wenn man von jemandem lernen kann, der besser ist als man selbst. Allerdings hat man ein Problem, wenn man selber der beste ist. Jedoch ist dies im Schach kein Problem. Schachcomputer haben uns seit ihrer 'Geburt' einiges beigebracht über Schach. Zum Beispiel hat AlphaZero den frühen Schub des h-Bauern popularisiert, welcher vor dessen Zeit noch als dubios angesehen wurde. Jedoch ist man heute kaum noch überrascht von früher h-Bauern schüben. Jedoch ist dieser Einfluss kaum irgendwo grösser als in den Eröffnungen. Während man früher noch alles selber analysieren musste, kann das heute der Computer schneller und besser machen. Dabei hat der Computer auch keine Vorurteile gegen komische Züge, welche von einem Menschen aus Prinzip abgelehnt werden würden.

2.2.4 negative Einflüsse von Schachcomputern auf das menschliche Spiel

Schachcomputer haben unser Verständnis von Schach auf ein deutlich höheres Niveau gebracht. Somit würde man denken, dass Engines nur positiv zu betrachten sind. Jedoch kann man auch argumentieren, dass Enginges Schach 'ruiniert' haben. Vor Engines war das schummeln im Schach viel weniger ein Thema. Allerdings haben Schachcomputer eine neue, leicht zugängliche Art des Schummels ermöglicht. Und damit kamen auch viele schlechte Geschichten. So wurde zum Beispiel 2015 ein georgischer Grossmeister erwischt mit einem Handy auf der Toilette[1]. Das grösste Problem beim Cheating sind jedoch nicht die Cheaters selber. Davon gibt es weniger als dass man meinen würde**. Das Problem liegt eher in der Angst dass jemand schummeln könnte. So wurde der amerikanische Grossmeister Hans Niemann von Magnus Carlsen des Cheatings beschuldet, ohne jedoch irgendwelche Beweise zu liefern. Jedoch ist dies kaum die schlimmste Geschichte, da 202X der ehemalige Weltmeister Vladimir Kramnik praktisch jede Person an der Weltspitze das cheaten vorgeworfen hat. Dies jedoch ohne irgendwelche Beweise. Aus diesem Grund wurde Kramnik 2026 dann von der FIDE für 2 Jahre gesperrt.[6]. Die Angst vor Cheating geht allerdings weiter als grundlose Vorwürfe. Da Turniere nun auch strengere anti-cheat Massnahmen vornehmen müssen. So gibt es etwa Metalldetektoren bei grossen Turnieren, zufällige Durchsuchungen oder man darf gar kein Handy in den Turniersaal mitnehmen.

2.3 Wie funktioniert ein Schachcomputer?

2.3.1 Die Problematik

Einem Computer das Schach beizubringen ist nicht leicht. Schach ist nämlich ein sehr komplexes Spiel mit vielen möglichen Stellungen. Nur schon nach 10 Zügen aus der Startstellung gibt es fast 70 Trillionen mögliche Stellungen. Nach 15 Zügen geht diese Zahl schon auf über 2 Quadrilliarden Stellungen[12]. Gleichzeitig kann Stockfish unter optimalen Bedingungen 'nur' eine Milliarde Stellungen pro Sekunde rechnen[7]. Dies mag zwar nach sehr viel klingen, allerdings ist es nichts wenn man die Anzahl Stellungen anschaut die Stockfish durchlaufen muss. Allerdings muss es gar nicht alle Stellungen anschauen. Denn es gibt Techniken, mit denen man den Arbeitsaufwand minimieren kann.

2.3.2 minimax

Schach ist ein zweispielerspiel. Das bedeutet, dass beim eigenen Zug man selbst die beste Option machen möchte. Gleichzeitig möchte das der Gegner jedoch auch machen. Somit müssen wir unsere Optionen maximieren während wir die gegnerischen Optionen minimieren.

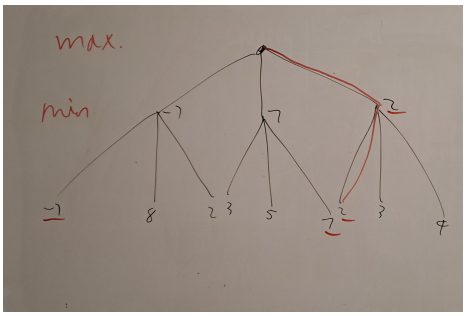


Abbildung 10: Minimax Baum

In diesem Beispiel hat es einen maximierenden Spieler und einen minimierenden Spieler. Der minimierende versucht immer an seinem Zug den minimalen Wert heraus zu bekommen. Gleichzeitig versucht der maximierende Spieler den maximalen Wert heraus zu bekommen. Somit muss der maximierende Spieler jenen Pfad wählen, wo der minimierende spieler das maximale minimum auswählen muss.[14].

2.3.3 alpha-beta Pruning

Alpha-Beta Pruning ist eigentlich eine verbesserte Version vom minimax Algorithus. Die Idee dahinter ist, dass man gewisse Sequenzen gar nicht anschauen muss, wenn man garantieren kann, dass es eine bessere Option gibt.

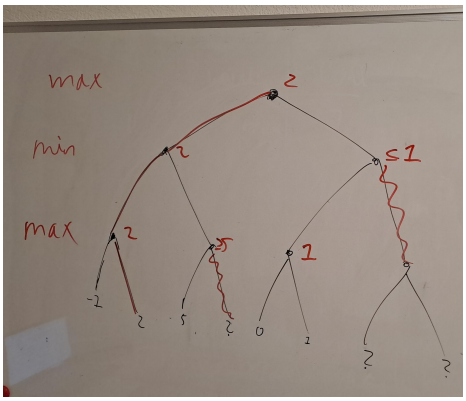


Abbildung 11: Enter Caption

In diesem Beispiel fangen wird links an. Dabei haben wir die Optionen -1 und 2. Da der maximierende Spieler dran ist, wählt er die 2 aus. Das 2 ist somit der momentan beste Wert. Wenn wir uns nun den zweiten Knoten des maximierenden Spielers anschauen, werden wir eine 5 finden. Da der maximierende Spieler nun an diesem Knoten mindestens eine 5 garantieren kann, können wir die Suche abbrechen. Dies liegt daran, dass der Knoten mindestens eine 5 als Wert haben wird. Da 5 grösser ist als 2 wird der minimierende Spieler somit niemals diesen Knoten auswählen und wir können somit die restlichen Züge ignoreren. Auf der rechten Seite schauen wir uns nun den dritten Knoten des maximierenden Spielers an. Dort finden wir zuerst eine 0 und danach eine 1. Der maximierende Spieler wählt somit die 1, wonach wir eine Stufe nach oben gehen. An diesem Knoten können wir nun garantieren, dass der minimierende Spieler maximal eine 1 bekommen wird. Somit müssen wir uns den Rest des Baumes gar nicht anschauen, da der maximierende Spieler in der Startstellung sich schon eine 2 garantieren kann auf der linken Seite. Somit ist die Suche beendet und die 2 ist der beste Weg für den maximierenden Spieler.

Beim alpha-beta Pruning ist es von grossem Vorteil, wenn man schnell den besten Zug findet. Deshalb muss man in Bezug auf einen Schach Computer die Reihenfolge der anzuschauenden Züge so ordnen, dass der Algorithmus zuerst jene Züge anschaut, die eine bessere Chance haben, gut zu sein. Im besten Fall findet man den besten Pfad sofort, wonach man viele Äste prunen kann. Im schlimmsten Fall findet man den besten Pfad als letztes, sodass man nichts prunen kann. Wichtig zu erwähnen ist noch, dass alpha-beta prunging das Resultat einer puren minimax Suche nicht verändert. Es beschleunigt sie nur.

Wenn man annimmt, dass nach jedem Zug die Anzahl möglicher Züge konstant ist, dann muss man b^n Knoten anschauen, wobei n die Tiefe ist und b die Anzahl Knoten nach einem Zug. Mit alpha-beta pruning und perfekter Sortierung muss man jedoch nur $b^{ceil[\frac{n}{2}]} + b^{floor[\frac{n}{2}]} - 1$ Knoten anschauen. Das ist eine massive Verbesserung. Jedoch funktioniert es, wie schon gesagt, nur mit perfekter Sortierung. Im schlimmstmöglichen Fall muss man auch b^n Knoten

anschauen, was einfach einer minimax Suche entspricht.

In der Praxis werden das Pruning durch zwei Variablen durchgeführt. Diese sind alpha und beta. Alpha stellt dabei den grösstmöglichen Wert dar der der maximierende Spieler bis zu einem bestimmten Punkt garantiern kann. während beta den tiefstmöglichen Wert an dem Punkt darstellt, der der minimierende Spieler garantieren kann. Wenn beta =¡ alpha ist, kann man den Pfad prunen, da die Spieler nie auf diesen Abschnitt gehen würden. Als Code würde dies so aussehen:

```
function minimax(position, depth, alpha, beta, maximizingPlayer)
   if depth == 0 or game over in position
      return static evaluation of position

   if maximizingPlayer
      maxEval = -infinity
      for each child of position
         eval = minimax(child, depth - 1, alpha, beta, false)
         maxEval = max(maxEval, eval)
         alpha = max(alpha, eval)
         if beta <= alpha
            break
      return maxEval

   else
      minEval = +infinity
      for each child of position
         eval = minimax(child, depth - 1, alpha, beta, true)
         minEval = min(minEval, eval)
         beta = min(beta, eval)
         if beta <= alpha
            break
      return minEval
```

Abbildung 12: Alpha-Beta algorithmus als Code von Sebastien lague aus:https://www.youtube.com/watch?v=l-hh51ncgDI

2.3.4 Stellungsbewertung

Damit die Suchalgorithmen von oben überhaupt funktionieren können, brauchen sie einen Weg, um Stellungen numerisch voneinander unterscheiden zu können. Diese numerische Einteilung macht die Stellungsbewertung. Sie analysiert eine Stellung nur aufgrund ihrer statischen Eigenschaften und gibt ihr dann einen Wert. Dieser Wert wird typischerweise in centipawns wiedergegeben, wobei ein positiver Wert Vorteil für weiss entspricht und ein negativer Wert Vorteil für schwarz. Somit macht es auch Sinn, dass man die weisse und schwarze Stellungen separat bewertet und dann voneinander subtrahiert. Dabei werden verschiedene Parameter benutzt, wie etwa Material, Königssicherheit, Figurenmobilität und noch viel mehr. Im praktischen Teil werde ich anhand eines konkreten Beispiels

erläutern, welche Parameter ich gewählt habe[8].

2.3.5 dynamische Suche

Die oben genannten Suchalgorithmen funkionieren nur bis zu einem Punkt gut. Das Problem liegt dabei an ihrer festgelegten Tiefe. Es gibt nämlich gewisse Stellungen, bei welchen es durchaus vorteilhaft ist, wenn man keinen festen cutoff hat. Dies nennt man den horizont Effekt[10]. Davon sind Stellungen betroffen, die gerade in einer forcierten Sequenz sind. Eine Sequenz ist forciert, wenn der letzte Zug des Gegners etwas geschlagen hat oder ein Schach war. Die Sequenz ist auch forciert, wenn der letzte eigene Zug ein Schach war. Im Verlauf meines Projektes habe ich selber realisiert, dass eine solche dynamische Suche hilfreich sein kann. Dabei habe ich erst später herausgefunden, dass diese Art der Suche schon etabliert ist unter dem Namen Quiescence search[13] bekannt ist.

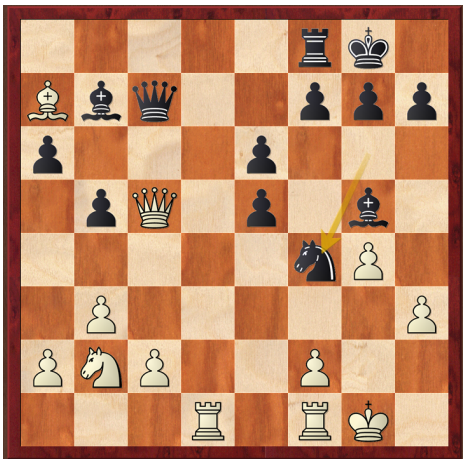


Abbildung 13: Enter Caption

Diese Stellung ist ein Beispiel dafür. Wenn wir dem Computer nun die Tiefe 3, also einmal weiss, dann schwarz, dann nochmals weiss, geben würden, würde er Dxc7 spielen. Dies liegt daran, dass nach der Sequenz 1.Dxc7 Se2+ 2.Kh2 Der Computer einfach aufhört. Wenn wir nun jedoch sagen, dass er weiterrechnen soll, wenn die Sequenz forciert ist, würde er in der Schlussstellung den Zug 2.Lf4# finden und würde dann die Dame nicht nehmen.

Dabei ist jedoch noch zu bemerken, dass diese dynamische Suche das eigentliche Problem nur nach hinten verschiebt. Man kann nicht jede forcierte Sequenz beliebig weit rechnen lassen. Würde man dies tun, müsste man mehrere Sequenzen berechnen, die hunderte Züge lang nur schachs sind. Somit muss man auch hier eine feste Obergrenze setzen. Da man jedoch nur forcierte Züge anschaut muss man weniger Stellungen anschauen. Somit kann man die Obergrenze weiter hoch schieben.

3 praktischer Teil

Beim praktischen Teil werde ich mein Vorgehen erklären und erläutern, wie mein Bot funktioniert und wie ich ihn aufgebaut habe

3.1 Vorgehen/Dokumentation

3.1.1 Versionenvergleich

Im Verlauf meiner Arbeit musste ich viele Versionen vergleichen. Dabei habe ich mithilfe von Cutechess die beiden Versionen sehr viele Spiele spielen lassen. Meist habe ich auf Tiefe 2 10000 Spiele durchgemacht. Allerdings habe ich bei manchen Tests auch nur 1000 Spiele gemacht, dafür aber Tiefe 4.

3.1.2 Datenauswertung, α, Score

Um diese Spiele dann auszuwerten, habe ich vorallem zwei Grössen verwendet. Diese sind der score und das Signifikanzniveau α

$score = \frac{wins+0.5*draws}{AnzahlPartien}$[11]

α wird wie folgt berechnet:

$\alpha = \Phi(\frac{wins-losses}{\sqrt{wins+losses}})$ α gibt an, wie warscheinlich es ist, dass der eine Bot stärker ist als der andere. Dabei werden üblicherweise Resultate zwischen 5% und 1% als signifikant betrachtet. Für meine Arbeit werde ich ein Resultat als signifikant betrachten ab 5%/95% oder z = 1.65 [**Signifikanz**]. z ist dabei der Term in der Klammer.

3.1.3 AI als Hilfsmittel

3.2 Das Schachbrett

Der Wohl wichtigste Teil des Schachs ist das eigentliche Brett. Dafür habe ich drei wichtige Darstellungsformen verwendet, wobei alle ihre eigenen Nutzen haben.

Objekt

Am meisten verwendete ich das Brett als Objekt. Das Brett selber ist dann ein Vektor. Dabei steht jedes Feld für ein Element des Vektors. Die Nummerierung beginnt dabei von weiss aus gesehen oben links, also a8 und geht dann der Reihe nach durch bis nach h1. a8 entspricht somit dem Index 0 und h1 entspricht dem Index 63.

	a	b	c	d	e	f	g	h
8	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
7	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
6	**16**	**17**	**18**	**19**	**20**	**21**	**22**	**23**
5	**24**	**25**	**26**	**27**	**28**	**29**	**30**	**31**
4	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**
3	**40**	**41**	**42**	**43**	**44**	**45**	**46**	**47**
2	**48**	**49**	**50**	**51**	**52**	**53**	**54**	**55**
1	**56**	**57**	**58**	**59**	**60**	**61**	**62**	**63**

Dazu kodieren verschiedene Buchstaben für unterschiedliche Figuren:

`'O'`: leeres Feld

`'P'`: Bauer (englisch Pawn)

`'N'`: Springer (englisch Knight)

`'B'`: Läufer (englisch Bishop)

`'R'`: Turm (englisch Rook)

`'Q'`: Dame (englisch Queen)

`'K'`: König

grosser Buchstaben → weisse Figur
kleiner Buchstaben → schwarze Figur

Der Springer bekommt dabei das 'N', da das K schon dem König überlassen wurde.

Der Vorteil an dieser Form ist, dass die Figurenzüge sehr praktisch zu beschreiben sind. Allerdings gibt es effizientere Formen, um das Brett darzustellen. Mehr dazu jedoch später. Dazu können nicht alle notwendigen Informationen in einem Vektor gespeichert werden, weshalb es wichtig ist, dass es ein Objekt ist. Dabei besitzt das Brett members zu jeder Art von Figuren und noch viel mehr

3.2.1 FEN

Eine weitere wichtige Darstellungsform war der FEN-code. Dieser ist nämlich der Gold-Standard wenn es um das Teilen von Schachstellungen geht. Aus diesem Grund war auch die erste Funktion die ich für meine Engine geschrieben habe tatsächlich eine FEN-Umwandlungsfunktion. Die Idee dahinter ist, dass die Stellung sowie alle wichtigen Zusatzinformationen wie Rochaderechte, mögliche en passents, wer am Zug ist und der wievielte Zug es ist in einen String gepackt werden.

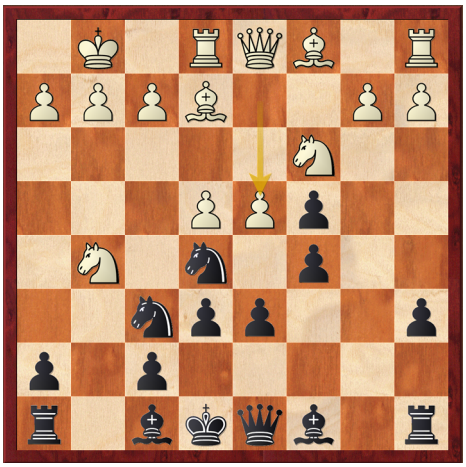


Abbildung 14: Mögliches en passent in der Partie Ciric - Rappazzo

Diese Stellung wird durch folgenden FEN-Code codiert: r1bqkb1r/5p1p/p2ppn2/2p1n1N1/2pPP3/2N5/PP2
b kq d3 0 11

Der erste Teil des Strings codiert die Stellung selber. Die '/' trennen dabei Reihen voneinander. Man beginnt in der von weiss aus gesehen oberen linen Ecke, also a8. Dabei notiert man weisse Figuren mit dessen korrespondierenden Grossbuchstaben während schwarze Figuren kleinbuchstaben bekommen. Die Zahlen stehen dabei für die Anzahl leeren Felder zwischen den Figuren oder Reihenumbrüchen.

Am Ende des Strings kann man die folgenden Informationen ablesen. das 'b' steht für 'schwarz ist am Zug'. Das 'kq' steht dafür, dass schwarz noch auf den Königsflügel sowie den Damenflügel rochieren darf. Hätten beide Seiten noch alle Rochaderechte, würde dort 'KQkq stehen. Das 'd3' steht für das Zielfeld des schlagenden Bauern[9].

3.2.2 Hashing

Die letzte Form ist die gehashte Form. Diese wird für Zuguumstellungstabellen verwendet. Da diese allerdings erst später relevant werden, werde ich sie erst dann erklären.

3.3 Zuggeneration

Meine Engine generiert zuerst alle pseudo-legalen Züge. Das heisst, alle Züge ohne zu berücksichtigen ob sie aufgrund von möglichen Schachs nicht gehen. Danach führt es für jeden pseudo-legalen Zug einen Check durch, ob nach diesem Zug der eigene König im Schach ist. Wenn dies nicht der Fall ist, kommt das Startfeld dieses Zuges in eine Liste und das Zielfeld dieses Zuges in eine andere Liste. Dies ist praktisch, da aufgrund der Vektor eigenschaft des Brettes man nur das Feld als Zahl, bzw. Index angeben muss. Somit geben die korrespondierenden Indizes des Startfeldvektors und des Zielfeldvektors einen Zug. problematisch wird dieses Vorgehen erst bei der Umwandlung, da diese vier Züge in einem sind. Allerdings habe ich auch hierfür eine Lösung gefunden. Diese wird jedoch separat im Suchalgorithmus behandelt

3.4 Stellungsbewertung

Ich habe im Abschnitt 2.3.4 dieses Konzept schon eingeführt. Nun will ich allerdings konkret zeigen, wie ich dieses implementiert habe und welche Parameter ich gewählt habe.

3.4.1 Spielende

Eine Partie Schach kann auf drei Arten ausgehen. Entweder gewinnt weiss, es gewinnt schwarz oder es endet in einem Unentschieden. In einer echten Partie kann man durch Schachmatt, Aufgabe oder auf Zeit gewinnen. Da der Bot jedoch nicht aufgeben kann und ohne Zeitdruck spielt, kennt er nur den Sieg durch Schachmatt. Dieser tritt ein, wenn eine Seite keine Züge mehr hat um den König aus dem Schach zu bringen.

```
if(board.legal_starting_squares.size() == 0){
    if(iswhiteincheck(board)){
        return -1000;
    }
    if(isblackincheck(board)){
        return 1000;
    }
    return 0;
}
```

Abbildung 15: Enter Caption

Da das Programm wie schon erwähnt eine Liste aller legalen Startfelder macht, kann man annehmen, dass das Spiel vorbei ist, wenn dieser Vektor leer ist. Dabei gewinnt schwarz, wenn weiss im Schach ist und umgekehrt. Für den Sieg wählte ich 1000, bzw. -1000 damit klar ist für den Bot, wann es vorbei ist. Die normale Stellungsbewertung kann diese Werte nie von alleine erreichen. Falls jedoch weder weiss noch schwarz im Schach stehen, muss ein Patt vorliegen, sodass es ein remis ist und 0 zurückgegeben wird.

3.4.2 Figurenwerte

Die eigentliche Stellungsbewertung beginnt mit dem Zählen des Materials selber. Dabei habe ich den Figuren folgende Werte zugewiesen:

Bauer = 1 Springer = 3 Läufer = 3.5 Turm = 5 Dame = 9

Traditionellerweise würde man dem Läufer auch den Wert 3 geben. Allerdings ist es heutzutage ein grösstenteils akzeptierter Fakt, dass Läufer meistens etwas besser sind als Spinger. Am Anfang des Programms habe ich eine Variable Eval intitalisiert. Dabei addiere ich alle weissen Figuren und subrahiere alle schwarzen.

3.4.3 Springerfelder

Springer sind nicht überall gleichgut. Je näher am Zentrum sie sind, desto besser sind sie. Dafür habe ich folgende Tabelle entworfen:

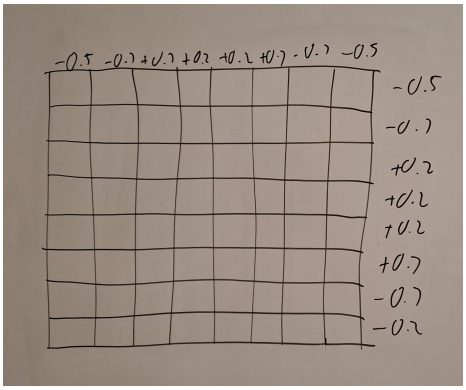


Abbildung 16: Enter Caption

Die Idee dahinter ist von sebastien Lauge[**seblague**]. Diese Tabelle ist für den weissen Springer. Die für den schwarzen Springer ist genau gleich, mit der Ausnahme der ersten und letzten Reihen.

3.4.4 Springervorposten

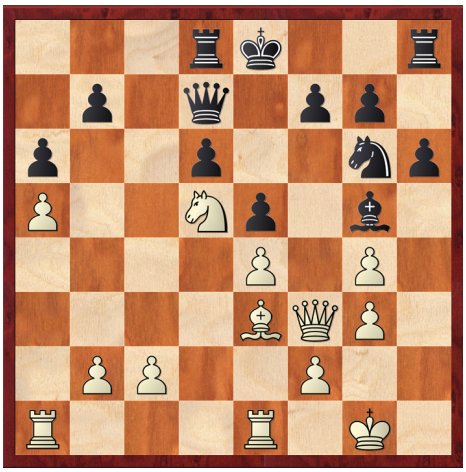


Abbildung 17: Vorposten auf d5

Ein Vorpostenfeld für einen Springer ist ein Feld, auf dem der Springer von keinem gegnerischen Bauern angegriffen werden kann. Konkret heisst das, dass es auf den benachbarten Linien vor dem Springer keine Bauern haben dar. Dies habe ich mit einem Bonus von 0.1 belohnt. Wenn der Springer zusätzlich noch von einem eigenen Bauern gedeckt ist, bekommter nochmals einen Bonus von 0.1.

3.4.5 Freibauern

Ein Freibauer ist ein Bauer, der keine gegnerische Bauern auf der eigenen oder auf den benachbarten Linien vor sich hat.

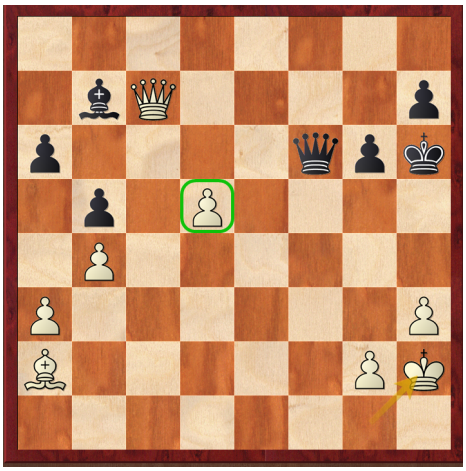


Abbildung 18: Der Bauer auf d5 ist ein Freibauer

Für die Bewertung von Freibauern habe ich folgende Formel entworfen:
$Bonus = 0.1 * (8 - Reihe)^2 * \frac{1}{AnzahlFiguren+1}$
Dies gilt allerdings nur für weisse Bauern. Schwarze Bauern haben im quadrierten Term nur 'Reihe', da das gehen gegen die weisse Grundreihe den Reihen Term wachsen lässt.

Die Idee dahinter ist, dass Freibauern enorm viel wertvoller werden sollten je näher sie an der Grundreihe sind. Dazu sollten sie auch wertvoller sein, je weniger Figuren auf dem Brett sind. Dabei wird unter der Anzahl Figuren alle Figuren ausser Bauern und natürlich ohne die Könige berücksichtigt. Dabei ist es wichtig, dass diese Anzahl um eins addiert wird, da sonst bei nur Königen durch 0 geteilt werden würde. Den konstanten Faktor 0.1 hatte ich einmal auf 0.5 gehabt. Allerdings hat die Engine dann Freibauern als viel zu wertvoll eingeschätzt. Ein weisser Bauer ein Feld von der Umwandlung entfernt hat einen Reihenbonus von 49. Dies liegt daran, dass er vom Computer aus gesehen auf der Reihe 1 ist. Wenn dann der konstante Faktor nur 0.5 ist, ist der Bonus schon auf 24.5. Wenn nun auch nur noch vier Figuren auf dem Brett sind, wäre der Bonus schon auf 4.9, fast so viel Wert ist wie ein Turm. Dies ist natürlich viel zu hoch. Wenn wir nun dieselbe Situation mit der jetzigen Formel betrachten, wäre die Rechnung
$0.1 * (8 - 1)^2 * 1/5 = 0.98$
Was ein viel angemessenerer Bonus ist.

Dabei geht es vorallem darum, dass noch Figuren auf dem Brett sind. Ein Bauer auf der siebten Reihe mit nur noch Königen auf dem Brett würde auch einen Bonus von 4.9 auslösen, jedoch macht dieser Bonus mit nur noch Königen auf dem Brett dann durchaus Sinn.

3.4.6 Matthilfe

Im Verlauf des Entwickeln des Bots hatte ich viele Probleme. Allerdings hat mich enorm überrascht, wie sehr der Bot Probleme hat, einen König matt zu setzen. Selbst mit deutlich mehr Material.

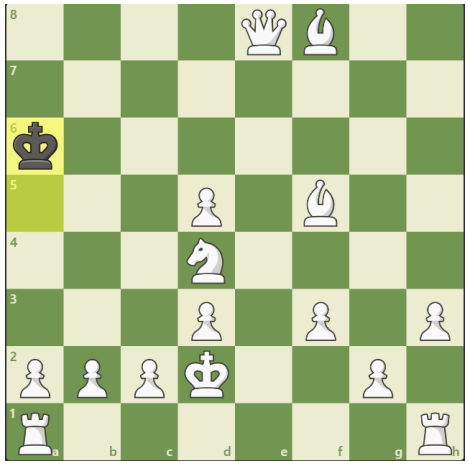


Abbildung 19: Trotz dem 36 Punkte Vorsprung schaffte mein Bot hier nur ein remis mit weiss

Das Problem liegt darin, dass das Schachmatt selbst nicht in der sichtbaren Tiefe des Bots liegt. Somit muss man dem Bot helfen, damit er gewonnene Stellungen tatsächlich gewinnt.

```
if(eval > 8){
        eval += (8-onecounter(board.blackKing[0].legal_destinations))/8.0; //bei gewonnen Stellungen sollte der König eingeengt werden
        // CHANGE 8 (speed): BEFORE: if(piececounter(board.board) <= 4){  NOW: uses piece_count (counted once above)
        if(piece_count <= 4){ //Bei gewonnenen Endspielen sollte der Abstand zum gegnerischen König minimiert werden sowie dessen Distanz zu der Ecke.
            eval -= sqrt(((board.whiteKing[0].square / 8) - (board.blackKing[0].square / 8)) * ((board.whiteKing[0].square / 8) - (board.blackKing[0].square / 8)) + ((board.whiteKing[0].square % 8 - board.blackKing[0].square % 8)) * ((board.whiteKing[0].square % 8 - board.blackKing[0].square % 8))); //Distanz zum gegerischen König
            if(piece_count > 1){
                eval += 1.0/(abs(board.whiteKing[0].square / 8 - 4)+1) + 1/(abs(board.whiteKing[0].square % 8 - 4)+1); //Distanz zur Ecke
            }
        }
    }
```

Diese Bedingung ist für gewonnene Stellungen von weiss. Dabei bekommt weiss einen Bonus für jedes Feld, das er vom schwarzen König wegnimmt. Sobald dann Stellungen mit weniger als fünf Figuren gewonnen sind, wird eine separate Bedingung aktiviert. dabei sollte der Abstand zum gegnerischen König minimiert werden. Zuletzt gibt es noch noch einen Bonus, wenn der König möglichst im Zentrum ist. Dabei funktioniert dieser Bonus nicht, wenn nur noch eine Figur auf dem Brett ist. Die Idee dahinter ist, dass man den König braucht, um mit einer Figur matt zu setzen.

3.4.7 König im Mittelspiel/Endspiel

Die Rolle des Königs im Schach ist eine durchaus spezielle Rolle. Während der König im Mittelspiel oft anfällig ist und möglichst hinter Bauern versteckt werden sollte, wird er im Endspiel plötzlich zur wichtigsten Figur, da man mit dem König effizient Bauern jagen kann. Aber ab wann sollte man den König als aktive Figur verwenden und wann ist es am besten, wenn man ihn sicher behält?

```
1  if((piece_count > 6 && board.whiteQueen.size() != 0 && board.blackQueen.size() !=
   ↪  0) || (piece_count > 10 && board.whiteQueen.size() == 0 &&
   ↪  board.blackQueen.size() == 0)){
2      if(board.whiteKing[0].square == 63 || board.whiteKing[0].square == 62 ||
       ↪  board.whiteKing[0].square == 58 || board.whiteKing[0].square == 57 ||
       ↪  board.whiteKing[0].square == 56){
3          eval += 1;
4      }
5      if(board.blackKing[0].square == 1 || board.blackKing[0].square == 2 ||
       ↪  board.blackKing[0].square == 0 || board.blackKing[0].square == 6 ||
       ↪  board.blackKing[0].square == 7){
6          eval -= 1;
7      }
8      eval -= onecounter(board.whiteKing[0].legal_moves) / 8;
9      eval += onecounter(board.blackKing[0].legal_moves) / 8;
10 }
11 // CHANGE 8 (speed): BEFORE: if(piececounter(board.board) <= 2 &&
   ↪  board.whiteQueen.size() == 0){  NOW: uses piece_count (counted once above)
12 if(piece_count <= 2 && board.whiteQueen.size() == 0){ //maximiert Königsmobilität
   ↪  im Endspiel
13     eval += 1/onecounter(board.blackKing[0].legal_destinations) / 8;
14     eval -= 4/(abs(board.blackKing[0].square / 8 - 4)+1);
15     for(pawn &Pawn : board.whitePawns){ //sollte die Distanz zwischen König und
       ↪  dem nächsten unverteidigtwn Bauern minimieren.
16         if(!square_attacked(board.board, Pawn.square, 1)){
17             if(distance(board.blackKing[0].square, Pawn.square) < dist){
18                 dist = distance(board.blackKing[0].square, Pawn.square);
19             }
20         }
21     }
22     eval -= 1/sqrt(dist);
23 }
```

Mit diesem Codeabschnitt habe ich dieses Problem gelöst. Dabei ist dies keine perfekte Lösung und es bringt auch seine eigenen Probleme mit sich. Wenn mehr als sechs Figuren auf dem Brett sind, wobei zwei davon Damen sind, oder mehr als 10 Figuren auf dem Brett sind ohne die Damen, wird jene Bedingung aktiviert, bei der der König in Sicherheit bleiben sollte. Ist diese Bedingung nicht erfüllt, passiert vorerst nichts. Bis dann nur noch zwei Figuren auf dem Brett sind. Dabei darf weiss seine Dame nicht mehr haben. Der hier gezeigte Abschnitt gilt für den schwarzen König. Nachher wird die Mobilität des eigenen Königs gefördert. Er sollte auch ins Zentrum kommen. Dazu sollte er so nahe wie möglich an jene gegnerischen Bauern gehen, die nicht gedeckt sind.

3.4.8 Nebenparameter

Während manche Parameter längere Erklärungen brauchen, sind hier diejenigen Parameter besammelt, die man in eins bis zwei Sätzen zusammenfassen kann.

3.4.8.1 mögliche Schachs Ich gebe einen Bonus von 0.1 aus für jedes Schach, das eine Figur machen kann. Die Idee dahinter ist, dass der Bot dann versucht die Anzahl mögliche Schachs zu minimieren und somit versucht, den König sicher zu halten.

3.4.8.2 Läufer auf dem Startfeld Ich habe den Läufern einen Abzug von 0.2 gegeben, falls sie sich noch auf dem Startfeld befinden. Dies habe ich getan, damit der Bot sie entwickelt

3.4.8.3 Zentrumsbauern Für jeden Bauern, der sich im Zentrum befindet(e4, d4, e5, d5), gibt der Bot einen Bonus von der Anzahl Figuren/14 aus. Dies ist, da kontrolle über Raum unwichtiger wird, je weniger Figuren auf dem Brett sind.

```
1 if(pawn.square == 27 || pawn.square == 28 || pawn.square == 35 || pawn.square ==
  ↪  36){
2             eval -= piece_count / 14.0;
3         }
```

3.4.8.4 schwache Bauern Ein Bauer ist schwach, sobald

3.4.9 Mobilität

Bei der Mobilität wird ein kleiner Bonus ausgegeben für jedes Feld, das von einer Figur begehbar ist. Bei meinem Bot bekommt jede Figur einen Bonus von 0.05 pro Feld ausser die Dame. Sie bekommt nur 0.01. Und dies erst sobald mindestens zwei Figuren aubgetauscht wurden. Dies ist, damit der Bot nicht einfach die Dame am Anfang herausbringt. Ursprünglicherweise wollte ich jeder

Figur einen Bonus von 0.1 geben und der Dame einen Bonus von 0.05. Allerdings hat dies beim Test mit 10000 Partien gegen die Version ohne die Mobilität folgendes Resultat zurückgegeben:

```
 3398 - 3503 - 3099  [0.495] 10000
```

Abbildung 20: 10000 Partien auf Tiefe 2

Um dieses Resultat dann zu prüfen habe ich nochmals einen Test auf Tiefe 4 mit 1000 Partien durchgeführt. Dieser ging so aus:

```
BotA vs BotB: 325 - 309 - 366  [0.508] 1000
```

Abbildung 21: 1000 Partien auf Tiefe 4 zwischen der alten Mobilität und dem ohne Mobilitätsbonus

Bot A entspricht bei diesen Tests jeweils dem Bot mit der mobilität. Das α der beiden Tests ist dabei jewils 10.4% und 73.5%. Beide Resultate sind somit nicht signifikant. Nach diesen Tests habe ich den Mobilitäts auf die oben genannten Werte angepasst. Dabei habe ich den neuen Bot 10000 Partien gegen einen Bot ohne Mobilitätsbonus spielen lassen und 1000 Partien auf Tiefe 4 gegen den ursprünglichen Bot. der erste Test ging so aus:

```
BotA vs BotB: 4475 - 2870 - 2655  [0.580] 10000
```

Abbildung 22: Bot A hat die verbesserte Mobilität, BotB ist der Testbot ohne Bonus

α beträgt hier 1, sodass dieses Resultat als äusserst Signifikant einzustufen ist. Der zweite Test gegen den ursprünglichen Bot ging dabei wie folgt aus:

```
BotA vs BotB: 263 - 430 - 307  [0.416] 1000
```

Abbildung 23: Bot A ist der ursprüngliche Bot, Bot B der verbesserte

Das α von BotB ist hier wieder 100%. Somit kann man klar sagen, dass eine deutliche Verbesserung in der Mobilität erreicht wurde.

3.4.10 Wichtigkeit der Parameter

Für diesen Abschnitt habe ich den Bot entweder 1000 oder 2000 Partien gegen sich selber spielen lassen. Wie genau die Bots gegeneinander spielen wird später noch erklärt Dabei habe ich immer jeweils einen parameter entfernt. Hier sind die scores:

In der Zeile Test habe ich zwei identische Bots gegeneinander spielen lassen, um zu prüfen, ob alles funktioniert. Dabei sollte der score nahe an 0.5 sein, was er auch ist.

Parameter	score	Partien	Tiefe	α	Signifikant
Test	0.496	2000	2	0.33	nein
Material	0.995	1000	2	1	Ja
Springerfelder	0.497	1000	2	0.41	nein
Schach Bonus	0.500	1000	2	0	nein
Freibauern	0.645	10000	2	1	ja
Mobilität	0.580	10000	2	1	ja
Vorposten	0.501	10000	2	0.56	nein
Zentrumsbauern	0.496	10000	2	0.18	nein
Läuferwert	0.496	10000	2	0.15	nein

Tabelle 1: Winrate des Bots ohne die Bewertungsfunktionen

3.4.11 Korrekturen der Evaluationsfunktion

weiss nicht, wie wichtig In diesem Abschnitt beschreibe ich korrekturen in der Stellungsbewertung, die ich versucht habe. Dabei teste ich die alten Versionen gegen die neuen. Die Verbesserung selber wird im entsprechenden Teil besprochen.

Parameter	score	Partien	Tiefe	α	Signifikant
Mobilität	0.584	1000	4	100%	ja
Königsabstand	0.621	10000	2	100%	ja

Tabelle 2: Verbesserungen

3.5 Suchalgorithmus

3.5.1 Benchmarks

3.6 wichtige Codeabschnitte

4 der Bot im Vergleich

4.1 Bot vs Bot

4.2 Partien gegen Bots

4.3 Partien gegen Menschen

4.4 Die Besten Spiele des Bots

4.5 Die Stärken und Schwächen des Bots

4.6 Eloeinschätzung

4.7 Verbesserungsmöglichkeiten

5 Diskussion

6 Fazit

7 Literaturverzeichnis

Literatur

[1] Peter Doggers. *Grandmaster Caught Cheating, Banned From Dubai Open*. URL: https://www.chess.com/news/view/grandmaster-caught-cheating-banned-from-dubai-open-2319.

[2] Mike Klein. *Google's AlphaZero Destroys Stockfish In 100-Game Match*. URL: https://www.chess.com/news/view/google-s-alphazero-destroys-stockfish-in-100-game-match.

[3] Stefan Klein. *Wie berechenbar ist das Schachspiel*. 2011. URL: https://www.sfbux.de/wp-content/uploads/artikel/berechenbarkeit.pdf.

[4] Eric van Reem. *Der Traum vom Computerschach Eine kleine Geschichte des Computerschachs*. URL: http://www.scrkuppenheim.de/coko/computerschach.htm.

[5] Stockfish. *Stockfish Docs - Classical versus NNUE Evaluation*. URL: https://official-stockfish.github.io/docs/stockfish-wiki/Advanced-topics.html#classical-versus-nnue-evaluation.

[6] Tarjei J. Svensen. *FIDE Bans Former World Champion Vladimir Kramnik For 2 Years*. URL: https://www.chess.com/news/view/fide-bans-kramnik-for-two-years.

[7] tbd. *tbd*. URL: https://chessify.me/blog/nps-what-are-the-nodes-per-second-in-chess-engine-analysis.

[8] Chess Programming Wiki. *Evalutation*. URL: https://chessprogramming.org/Evaluation.

[9] Chess Programming Wiki. *Forsyth-Edwards Notation*. URL: Forsyth-Edwards%20Notation.

[10] Chess Programming Wiki. *Horizon Effect*. URL: https://chessprogramming.org/Horizon_Effect.

[11] Chess Programming Wiki. *Match Statistics*. URL: https://chessprogramming.org/Match_Statistics.

[12] Chess Programming Wiki. *Perft results*. URL: https://chessprogramming.org/Perft_Results.

[13] Chess Programming Wiki. *quiescence search*. URL: https://chessprogramming.org/Quiescence_Search.

[14] Chess Programming wiki. *Minimax*. URL: https://chessprogramming.org/Minimax.